

Computer Graphics

Professor YoungWoon Cha

Computer Science and Engineering
Metaverse Convergence
Graduate School

Contents

- Assignment 4
- Questions
- Instructions



Assignment 4

Improving the Ray Tracer

- Objective
 - The goal of this assignment is to improve the previous ray tracer and shading model with post-processing methods.
- Ray Tracer Specification (so far)
 - Scene configuration
 - Shading configuration
 - Code architecture
- Implementing Post-Processing Methods
 - Gamma Correction
 - Sampling-based Anti-aliasing



Scene Configuration

- The scene consists of the following four objects:
 - Plane P, with equation $y = -2$.
 - Sphere S1, with center at $(-4, 0, -7)$ and radius 1.
 - Sphere S2, with center at $(0, 0, -7)$ and radius 2.
 - Sphere S3, with center at $(4, 0, -7)$ and radius 1.
- Camera
 - Assume a perspective camera
 - Eye point at $e = (0, 0, 0)$
 - Orientation given by $u = (1, 0, 0)$, $v = (0, 1, 0)$ and $w = (0, 0, 1)$.
 - (Note that the camera is looking along the direction $-w$.)
 - Assume that the viewing region on the image plane is defined by $l = -0.1$, $r = 0.1$, $b = -0.1$, $t = 0.1$, and $d = 0.1$.
 - Also assume that the image resolution is 1024×1024 (i.e., $n_x = n_y = 1024$).



Shading Configuration

- Use the scene and camera settings from Q1.
- Use the Phong model to shade the scene, with the following material parameters for each object:
 - Plane P: $k_a = (0.2, 0.2, 0.2)$, $k_d = (1, 1, 1)$, $k_s = (0, 0, 0)$, with specular power 0.
 - Sphere S1: $k_a = (0.2, 0, 0)$, $k_d = (1, 0, 0)$, $k_s = (0, 0, 0)$, with specular power 0.
 - Sphere S2: $k_a = (0, 0.2, 0)$, $k_d = (0, 0.5, 0)$, $k_s = (0.5, 0.5, 0.5)$, with specular power 32.
 - Sphere S3: $k_a = (0, 0, 0.2)$, $k_d = (0, 0, 1)$, $k_s = (0, 0, 0)$, with specular power 0.
- Note that the k_a , k_d , and k_s values are specified as (red, green, blue) values.



Shading Configuration

- Assume a single **point light source** at $(-4, 4, -3)$, emitting white light with unit intensity and no falloff.
 - Light Intensity: $I = ?$
 - Light Direction: $\mathbf{l} = ?$
- Assume a white ambient light with unit intensity.
 - Ambient Light: $I_a = ?$
- The ray tracer stores the appropriate illumination values at each pixel, computed using the Phong shading model.
- Shadows from the point light are implemented using shadow rays.



Code Architecture

- Implement the following classes:
 - Ray
 - Camera
 - Surface
 - Plane (inherits from Surface)
 - Sphere (inherits from Surface)
 - Scene
 - Any additional classes as needed.



Code Architecture

- Implement the following functions:

```
for 0 <= iy < ny {  
    for 0 <= ix < nx {  
        ray = camera.getRay(ix, iy);  
        c = scene.trace(ray, 0, +inf);  
        image.set(ix, iy, c);  
    }  
}
```

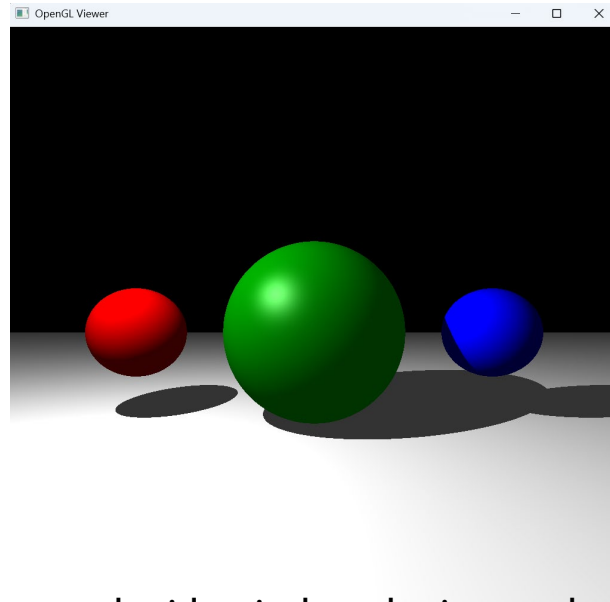
```
Scene.trace(Ray ray, tMin, tMax) {  
    surface, t = hit(ray, tMin, tMax);  
    if (surface is not null) {  
        point = ray.evaluate(t);  
        normal = surface.getNormal(point);  
        return surface.shade(ray, point, normal, light);  
    }  
    else return backgroundColor;  
}
```

```
Surface.shade(ray, point, normal, light) {  
    shadRay = getRay(point, light.pos - point);  
    if (shadRay not blocked) {  
        v = -normalize(ray.direction);  
        l = normalize(light.pos - point);  
        // compute shading  
    }  
    return black;  
}
```

Questions

Q1: RayTracer_v1

- Demonstrate the previous RayTracer implementation:
 - Use the results from HW3
 - Ray intersection
 - Phong shading
 - Code structure



Your output must be identical to the image shown here.

- Update your HW3 results if they differ from the provided figure:
 - If your HW3 output has any discrepancies compared to the result shown here (such as differences in color or specular shading radius), you must update your code so that your output exactly matches the image.

Q2: Gamma Correction

- Implementing Gamma Correction
 - Use the results from Q1
 - Perform gamma correction with $\gamma = 2.2$.
 - Output: RayTracer_v1 + Gamma Correction



Q3:Antialiasing

- Implementing a Sampling-based Anti-aliasing method
 - The image produced in Q2 contains unsightly “jaggies” at the boundaries of the spheres.
 - In this part, we will remove the jaggies using antialiasing.
 - Generate $N = 64$ samples per pixel by casting N random eye rays within the pixel’s area.
 - For each pixel, apply a box filter with an amplitude of $1/N$ and support equal to the extents of the “pixel rectangle” to estimate the pixel value.
 - The resulting image should have significantly reduced aliasing artifacts.
 - Use the results from Q2
 - Output: RayTracer_v1 + Gamma Correction + Antialiasing



Q4: Implement a New Feature

- Implement and demonstrate a custom feature
 - e.g., applying different anti-aliasing or post-processing methods (other than gamma correction or different sampling strategies)
 - e.g., provide detailed comparisons with different parameters (with various gamma values, or different sampling rates)
 - This question will be graded on a relative scale (A/B/C).
 - Fewer than 50% of students will receive an 'A' grade.



Instructions

Submission Guidelines

- Important Notes

- Your code must include a separate project for each question but with a single solution.
- You are not allowed to use OpenGL to perform hardware rasterization.
 - You must implement your own ray tracing and shading algorithms.
- Along with your code, please submit output screenshots for each question and explain them during your presentation.

- Submission Requirements

- Presentation video: Under 5 minutes and 30MB maximum (MP4 format)
- Presentation slides: PDF format
- Source code + output screenshots: VS 2022 project and image files (ZIP format)



Summary

- Assignment 4
- Questions
- Instructions



End of Class

Thank you

E-mail: youngcha@konkuk.ac.kr

